DÉMO 003-G · CHANGEMENTS ENTRE LES ÉTAPES

Formulaire

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1** · etape-1-template (3 fichiers, en entier)
2. **Étape 1 → 2** · etape-2-usestate (1 fichier)
3. **Étape 2 → 3** · etape-3-is-connected (1 fichier)
4. **Étape 3 → 4** · etape-4-zod (1 fichier)
5. **Étape 4 → 5** · etape-5-react-hook-form (1 fichier)

Étape 1 etape-1-template : point de départ

Le point de départ : le design de l'écran de login (HTML + CSS), sans logique

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (222 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg

package.json (gabarit Vite) : dépendances ajoutées : @hookform/resolvers ^5.2.2, react-hook-form ^7.68.0, zod ^4.2.1

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 9 lignes

```jsx
import "./App.css";
import Form from "./components/Form";

function App() {
  // Le formulaire de login : seul le fichier components/Form.jsx change d'une étape à l'autre
  return <Form />;
}

export default App;
```

DÉPART **src/components/Form.jsx** 28 lignes

```jsx
// Étape 1 (template) : l'écran de login, seulement l'interface (pas encore de state, ni de validation)
function Form() {
  return (
    <div className="container">
      <h1>Login Component</h1>
      <h2>Enter your login credentials</h2>
      <form name="login-form">
        <div className="form">
          <div className="form-group">
            <label>Email</label>
```

```
            <input type="text" />
          </div>

          <div className="form-group">
            <label>Password</label>
            <input type="password" />
          </div>

          <button className="btn" type="submit">
            Login
          </button>
        </div>
      </form>
    </div>
  );
}

export default Form;
```

Étape 1 → 2 etape-1-template → etape-2-usestate

Un state par champ (useState), onChange, e.preventDefault() et un console.log

MODIFIÉ **src/components/Form.jsx** +60 −5

```
@@ -1,21 +1,65 @@
- // Étape 1 (template) : l'écran de login, seulement l'interface (pas encore de state, ni de validation)
+ // Étape 2 : formulaire géré avec useState (un state par champ, onChange)
+ import { useState } from "react";
+
  function Form() {
+   const [email, setEmail] = useState("");
+   const [password, setPassword] = useState("");
+   const [isError, setIsError] = useState(false);
+
+   const emailChange = (event) => {
+     // event.target.value : ce que l'utilisateur vient de taper dans le champ
+     const { value } = event.target;
+     const isEmailValid = value.includes("@");
+     if (isEmailValid) {
+       setEmail(value);
+     }
+   };
+
+   const passwordChange = (event) => {
+     const { value } = event.target;
+     if (value.length > 0) {
+       setPassword(value);
+     }
+   };
+
+   const handleSubmit = (event) => {
+     // preventDefault : empêche le navigateur de recharger la page à l'envoi
+     event.preventDefault();
+     if (email && password) {
+       console.log("Connexion :", email, password);
+     } else {
+       setIsError(true);
+     }
+   };
+
    return (
      <div className="container">
        <h1>Login Component</h1>
        <h2>Enter your login credentials</h2>
-       <form name="login-form">
+       <form
+         name="login-form"
+         onSubmit={(e) => {
+           handleSubmit(e);
+         }}
+       >
          <div className="form">
            <div className="form-group">
              <label>Email</label>
-             <input type="text" />
+             <input type="text" onChange={(e) => emailChange(e)} />
            </div>

            <div className="form-group">
              <label>Password</label>
-             <input type="password" />
```

```
                53 +              <input
                54 +                type="password"
                55 +                onChange={(e) => {
                56 +                  passwordChange(e);
                57 +                }}
                58 +              />
17              59                </div>
18                -
                60 +            {isError && (
                61 +              <p className="error">Please enter valid email and password.</p>
                62 +            )}
19              63                <button className="btn" type="submit">
20              64                  Login
21              65                </button>
                   @@ -26,3 +70,14 @@
26              70    }
27              71
28              72    export default Form;
                73 +
                74 + /*
                75 + // Email (version correcte)
                76 + const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
                77 +
                78 + // Password: au moins 6 caractères
                79 + const passwordRegex = /^.{6,}$/;
                80 +
                81 + const isEmailValid = emailRegex.test(email);
                82 + const isPasswordValid = passwordRegex.test(password);
                83 + */
```

Étape 2 → 3 etape-2-usestate → etape-3-is-connected

On ajoute le state isConnected : connecté → « Welcome to private page »

MODIFIÉ **src/components/Form.jsx** +36 −30

```
@@ -1,10 +1,11 @@
- // Étape 2 : formulaire géré avec useState (un state par champ, onChange)
+ // Étape 3 : on ajoute isConnected : si l'email et le mot de passe sont remplis, on affiche la page privée
  import { useState } from "react";

  function Form() {
    const [email, setEmail] = useState("");
    const [password, setPassword] = useState("");
    const [isError, setIsError] = useState(false);
+   const [isConnected, setIsConnected] = useState(false);

    const emailChange = (event) => {
      // event.target.value : ce que l'utilisateur vient de taper dans le champ
@@ -26,7 +27,7 @@
      // preventDefault : empêche le navigateur de recharger la page à l'envoi
      event.preventDefault();
      if (email && password) {
-       console.log("Connexion :", email, password);
+       setIsConnected(true);
      } else {
        setIsError(true);
      }
@@ -36,35 +37,40 @@
      <div className="container">
        <h1>Login Component</h1>
        <h2>Enter your login credentials</h2>
-       <form
-         name="login-form"
-         onSubmit={(e) => {
-           handleSubmit(e);
-         }}
-       >
-         <div className="form">
-           <div className="form-group">
-             <label>Email</label>
-             <input type="text" onChange={(e) => emailChange(e)} />
+       {isConnected ? (
+         <div className="success">Welcome to private page</div>
+       ) : (
+         <form
+           name="login-form"
+           onSubmit={(e) => {
+             handleSubmit(e);
+           }}
+         >
+           <div className="form">
+             <div className="form-group">
+               <label>Email</label>
+               <input type="text" onChange={(e) => emailChange(e)} />
+             </div>

+             <div className="form-group">
+               <label>Password</label>
+               <input
+                 type="password"
+                 onChange={(e) => {
```

```
                    passwordChange(e);
                  }}
              />
            </div>
            {isError && (
              <p className="error">Please enter valid email and password.</p>
            )}
            {isConnected && <p className="success">Welcome to private page</p>}
            <button className="btn" type="submit">
              Login
            </button>
          </div>

          <div className="form-group">
            <label>Password</label>
            <input
              type="password"
              onChange={(e) => {
                passwordChange(e);
              }}
            />
          </div>
          {isError && (
            <p className="error">Please enter valid email and password.</p>
          )}
          <button className="btn" type="submit">
            Login
          </button>
        </div>
      </form>
        </form>
      )}
    </div>
  );
}
```

Étape 3 → 4 etape-3-is-connected → etape-4-zod

Validation avec Zod seul (safeParse), sans librairie de formulaire

MODIFIÉ **src/components/Form.jsx** +35 −46

```
@@ -1,32 +1,36 @@
-// Étape 3 : on ajoute isConnected : si l'email et le mot de passe sont remplis, on affiche la page privée
+// Étape 4 : validation avec Zod seul (safeParse), sans librairie de formulaire
 import { useState } from "react";
+import * as z from "zod";
+
+// Le schéma Zod : les règles que chaque champ doit respecter
+const formSchema = z.object({
+  email: z.string().email("Email invalide"),
+  // Deux règles sur le mot de passe, testées dans l'ordre :
+  // .min(1) : le champ ne doit pas être vide → message « Mot de passe obligatoire »
+  // .min(6) : au moins 6 caractères → message « Min 6 caractères »
+  password: z
+    .string()
+    .min(1, "Mot de passe obligatoire")
+    .min(6, "Min 6 caractères"),
+});

 function Form() {
-  const [email, setEmail] = useState("");
-  const [password, setPassword] = useState("");
   const [isError, setIsError] = useState(false);
   const [isConnected, setIsConnected] = useState(false);
-
-  const emailChange = (event) => {
-    // event.target.value : ce que l'utilisateur vient de taper dans le champ
-    const { value } = event.target;
-    const isEmailValid = value.includes("@");
-    if (isEmailValid) {
-      setEmail(value);
-    }
-  };
-
-  const passwordChange = (event) => {
-    const { value } = event.target;
-    if (value.length > 0) {
-      setPassword(value);
-    }
-  };

   const handleSubmit = (event) => {
     // preventDefault : empêche le navigateur de recharger la page à l'envoi
     event.preventDefault();
-    if (email && password) {
+
+    // récupérer les valeurs depuis le formulaire
+    const formData = new FormData(event.currentTarget);
+
+    const email = String(formData.get("email") ?? "");
+    const password = String(formData.get("password") ?? "");
+
+    // validation Zod sans try/catch
+    const result = formSchema.safeParse({ email, password });
+    if (result.success) {
       setIsConnected(true);
     } else {
       setIsError(true);
```

```
@@ -37,34 +41,30 @@
37 41       <div className="container">
38 42         <h1>Login Component</h1>
39 43         <h2>Enter your login credentials</h2>
   44 +
40 45         {isConnected ? (
41 46           <div className="success">Welcome to private page</div>
42 47         ) : (
43    -         <form
44    -           name="login-form"
45    -           onSubmit={(e) => {
46    -             handleSubmit(e);
47    -           }}
48    -         >
   48 +         <form name="login-form" onSubmit={handleSubmit}>
49 49             <div className="form">
50 50               <div className="form-group">
51 51                 <label>Email</label>
52    -               <input type="text" onChange={(e) => emailChange(e)} />
   52 +               {/* name obligatoire pour FormData */}
   53 +               <input name="email" type="text" />
53 54               </div>
54 55
55 56               <div className="form-group">
56 57                 <label>Password</label>
57    -               <input
58    -                 type="password"
59    -                 onChange={(e) => {
60    -                   passwordChange(e);
61    -                 }}
62    -               />
   58 +               {/* name obligatoire pour FormData */}
   59 +               <input name="password" type="password" />
63 60               </div>
   61 +
64 62               {isError && (
65    -               <p className="error">Please enter valid email and password.</p>
   63 +               <p className="error">
   64 +                 {"Please enter valid email and password."}
   65 +               </p>
66 66               )}
67    -             {isConnected && <p className="success">Welcome to private page</p>}
   67 +
68 68               <button className="btn" type="submit">
69 69                 Login
70 70               </button>
@@ -76,14 +76,3 @@
76 76   }
77 77
78 78   export default Form;
79    -
80    - /*
81    - // Email (version correcte)
82    - const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
83    -
84    - // Password: au moins 6 caractères
85    - const passwordRegex = /^.{6,}$/;
86    -
87    - const isEmailValid = emailRegex.test(email);
88    - const isPasswordValid = passwordRegex.test(password);
89    - */
```

Étape 4 → 5 etape-4-zod → etape-5-react-hook-form

Version finale : React Hook Form + Zod + zodResolver (register, handleSubmit, errors)

MODIFIÉ **src/components/Form.jsx** +34 −33

```
@@ -1,9 +1,12 @@
- // Étape 4 : validation avec Zod seul (safeParse), sans librairie de formulaire
- import { useState } from "react";
- import * as z from "zod";
+ // Étape 5 (version finale) : React Hook Form + Zod, reliés par zodResolver
+ // useForm : le hook qui gère le formulaire
+ import { useForm } from "react-hook-form";
+ import { z } from "zod";
+ // zodResolver : relie React Hook Form à Zod
+ import { zodResolver } from "@hookform/resolvers/zod";

  // Le schéma Zod : les règles que chaque champ doit respecter
- const formSchema = z.object({
+ const schema = z.object({
    email: z.string().email("Email invalide"),
    // Deux règles sur le mot de passe, testées dans l'ordre :
    // .min(1) : le champ ne doit pas être vide → message « Mot de passe obligatoire »
@@ -15,26 +18,19 @@
  });

  function Form() {
-   const [isError, setIsError] = useState(false);
-   const [isConnected, setIsConnected] = useState(false);
+   const {
+     register, // relie un champ au formulaire
+     handleSubmit, // valide, puis appelle onSubmit si tout est bon
+     formState: { errors, isSubmitSuccessful, isValid }, // les erreurs, envoi réussi, formulaire valide
+   } = useForm({
+     // zodResolver : React Hook Form valide les champs avec le schéma Zod
+     resolver: zodResolver(schema),
+     defaultValues: { email: "", password: "" },
+     mode: "onSubmit",
+   });

-   const handleSubmit = (event) => {
-     // preventDefault : empêche le navigateur de recharger la page à l'envoi
-     event.preventDefault();
-
-     // récupérer les valeurs depuis le formulaire
-     const formData = new FormData(event.currentTarget);
-
-     const email = String(formData.get("email") ?? "");
-     const password = String(formData.get("password") ?? "");
-
-     // validation Zod sans try/catch
-     const result = formSchema.safeParse({ email, password });
-     if (result.success) {
-       setIsConnected(true);
-     } else {
-       setIsError(true);
-     }
+   const onSubmit = (data) => {
+     console.log("Send to server:", data); // ici tu fais ton appel API
    };

    return (
```

```
@@ -42,28 +38,33 @@
42 38          <h1>Login Component</h1>
43 39          <h2>Enter your login credentials</h2>
44 40
45    -        {isConnected ? (
   41 +        {isSubmitSuccessful ? (
46 42            <div className="success">Welcome to private page</div>
47 43          ) : (
48    -          <form name="login-form" onSubmit={handleSubmit}>
   44 +          <form name="login-form" onSubmit={handleSubmit(onSubmit)}>
49 45              <div className="form">
50 46                <div className="form-group">
51 47                  <label>Email</label>
52    -                {/* name obligatoire pour FormData */}
53    -                <input name="email" type="text" />
   48 +                {/* register("email") : relie le champ à React Hook Form (valeur, onChange…)
   */}
   49 +                <input type="text" {...register("email")} />
   50 +                {errors.email && (
   51 +                  <p className="error">{errors.email.message}</p>
   52 +                )}
54 53                </div>
55 54
56 55                <div className="form-group">
57 56                  <label>Password</label>
58    -                {/* name obligatoire pour FormData */}
59    -                <input name="password" type="password" />
   57 +                <input type="password" {...register("password")} />
   58 +                {errors.password && (
   59 +                  <p className="error">{errors.password.message}</p>
   60 +                )}
60 61                </div>
61 62
62    -              {isError && (
63    -                <p className="error">
64    -                  {"Please enter valid email and password."}
65    -                </p>
   63 +              {Object.keys(errors).length > 0 && (
   64 +                <p className="error">Veuillez corriger les champs invalides.</p>
66 65              )}
   66 +
   67 +              {isValid && <p className="success">Form valid (ready to send)</p>}
67 68
68 69              <button className="btn" type="submit">
69 70                Login
```